

Fine Tuning LLMs

Advanced Natural Language Processing
Sukkur IBA University
Sher Muhammad Daudpota

In-context Learning

- *The information passed through in-context learning can be used to define the output format of the model's responses, as you've seen in the section "Influencing the Model's Response with In-Context Learning"*

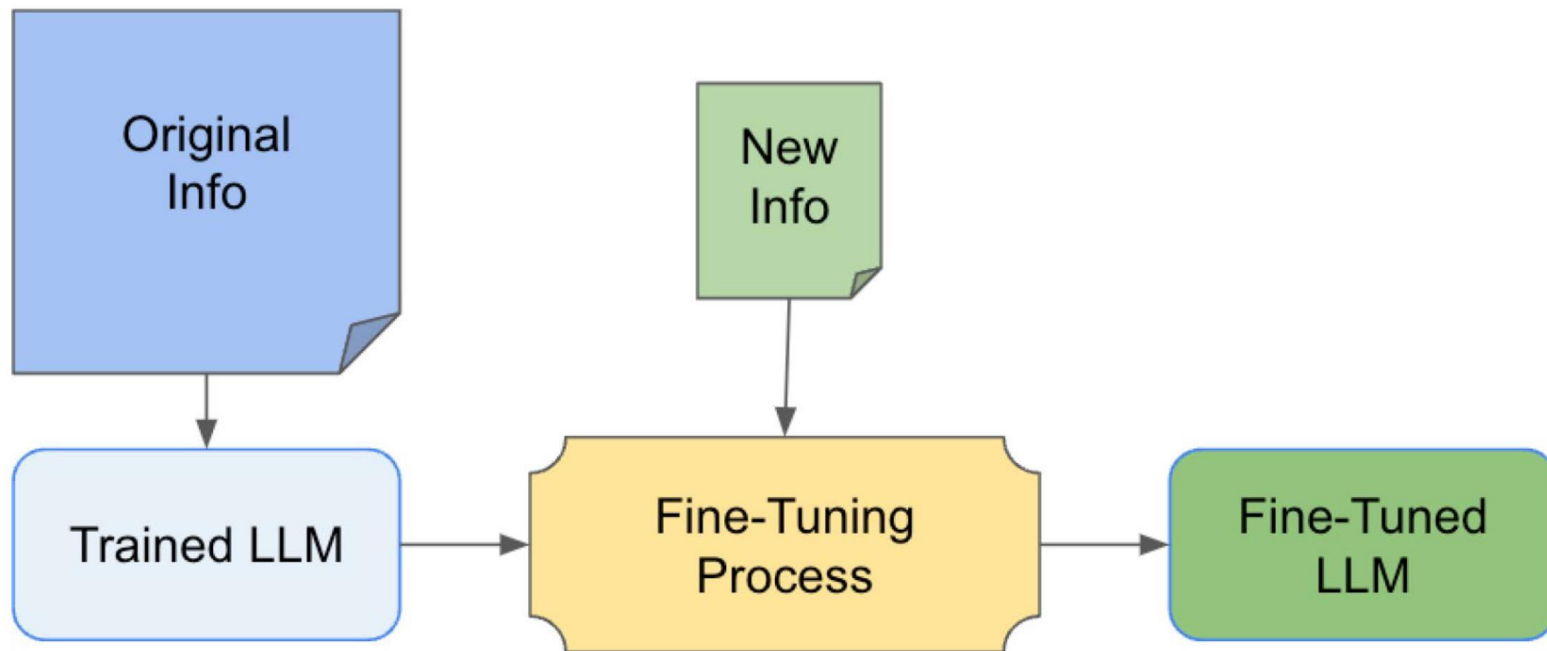
Concept of fine-tuning models

- Altering model behavior
 - In-context learning
 - RAG

Concept of fine-tuning models

- Fine-tuning is the process of adapting a previously trained model to specific tasks
 - *Can you recall when we used fine-tuning previously ????*
- This adaptation is achieved by conducting a second round of training with a smaller volume of specialized data.

Concept of fine-tuning models



Fine-Tuning and RAG

- Do they look same??

Fine-Tuning and RAG

- You might think that we're dealing with another way of influencing a model using external information, similar to the case of a RAG or even the information that can be passed in the prompt.
- It's similar, but not the same!!

Fine-Tuning and RAG

In this case, you're altering the model in such a way that it can never be the same again

Models Layers

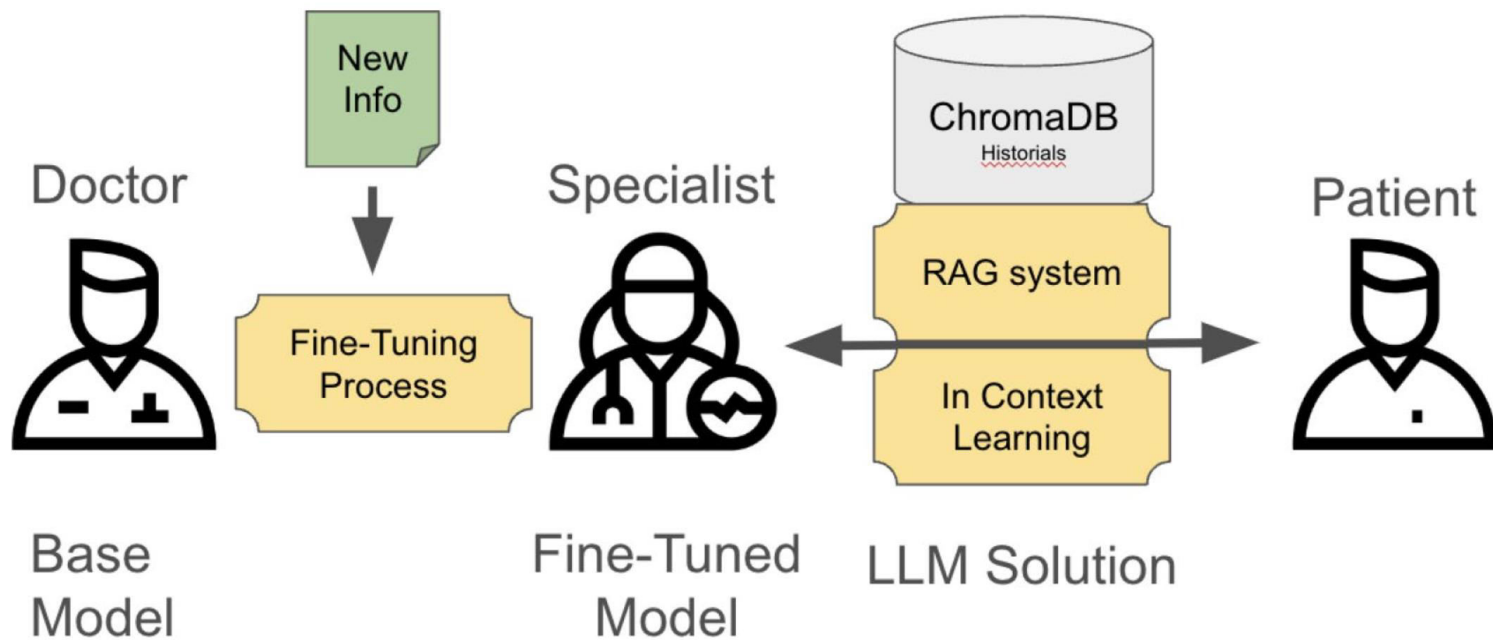
- **Embedding Layers:** Transform words into numerical representations, vectors, the model can understand. These vectors capture semantic information about sentences.
- **Transformer Layers** (Attention, Feedforward): The core of modern language models, these layers use mechanisms like self-attention to process all tokens and capturing relationships between them.
- **Output Layers:** This is the final layer that produces the model's predictions. For a language model, this could be the next word in a sentence, the sentiment of a piece of text, or the answer to a question.

Fine-tuning and Catastrophic forgetting

- Be careful about it...

When to use RAG and when to use Fine-Tuning

- Few possibilities...



When to use RAG and when to use Fine-Tuning

- In this setting, RAG or in-context learning contains,
 - Information that does not have to be fixed
 - Patient history
 - Recent studies
 - Diagnoses and treatments of patients with similar histories

When to use RAG and when to use Fine-Tuning

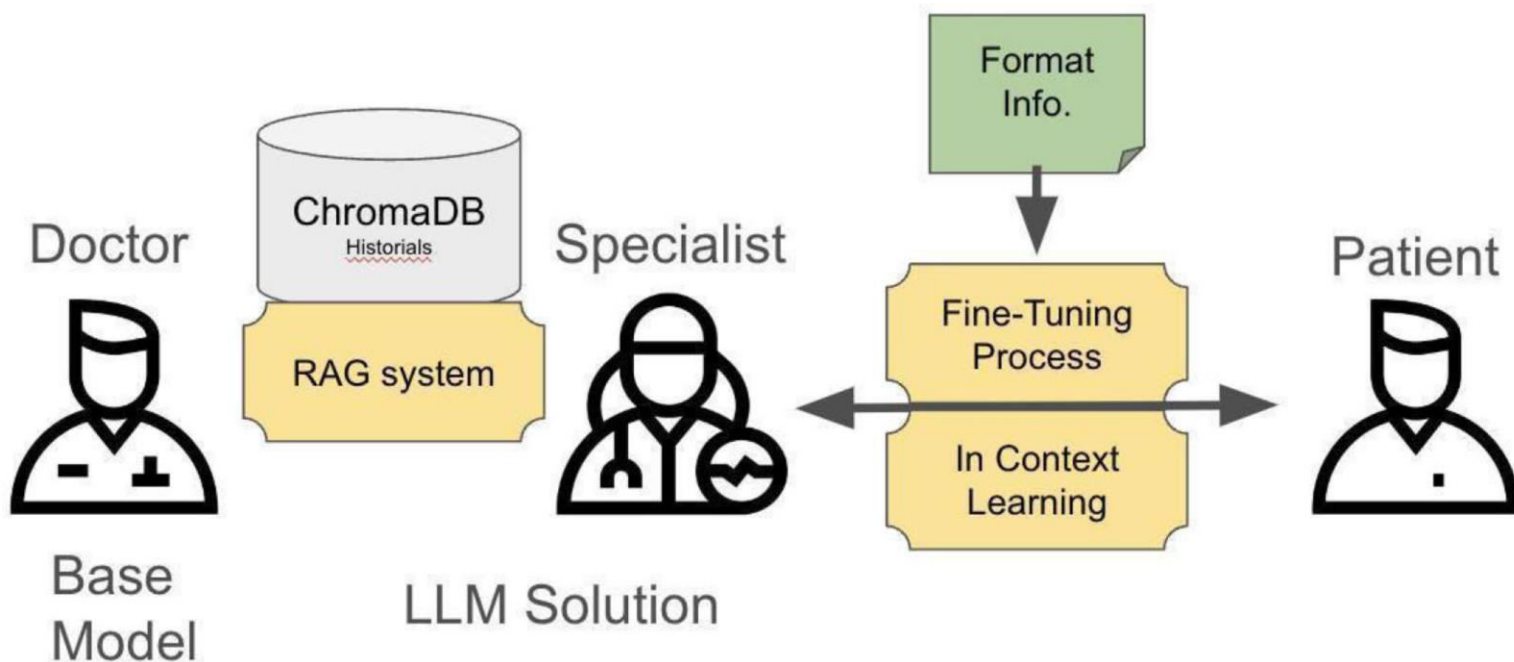
- In this setting, RAG or in-context learning contains,
 - Information that does not have to be fixed
 - Patient history
 - Recent studies
 - Diagnoses and treatments of patients with similar histories
- ***So, remember, that the information you provide to a model through fine-tuning cannot be removed. If you train it with data that turns out to be incorrect for any reason, you'll need to retrain the model from scratch with the refined data.***

When to use RAG and when to use Fine-Tuning

- The boundary between RAG and Fine-tuning is thin dotted line...
- Few recommendations,
 - If the information isn't permanent, don't include it into the model. Examples like restaurant menus, product prices, schedules, and other variable information should be fed through a RAG system.
 - All the information retrieved via RAG becomes part of the prompt. The longer the prompt, the more expensive it is to process. Fine-tuning a model doesn't have to be an expensive process, and depending on the use case, it might be cost-effective in the long run due to the reduction of necessary information in the prompt.

When to use RAG and when to use Fine-Tuning

- Nothing is absolute truth, an alternate school of thought,



LoRA – a PEFT

- **More Efficient Training:** It focuses on changing the weight of only a reduced number of parameters, reducing the required training time and resources.
- **Preserves Model Knowledge:** One of the problems with full fine-tuning is that by altering all parameters, it can lead to what is known as catastrophic forgetting, or catastrophic interference. LoRA reduces this possibility by keeping the majority of weights unchanged.

LoRA – a PEFT (Matrix Reduction)

M

1					50
50					

M (50X50)=2500

=

M = M1 * M2

1	2
50	

1					
2					50

M1(50x2)=100 M2(2x50)=100

LoRA – a PEFT (Intuition behind it)

- The idea:

“Don’t fine-tune the full big matrix, just learn two small matrices that represent a low rank change”

LoRA – a PEFT

For a linear layer in a transformer:

$$h = Wx$$

where

- $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$
- x is the input vector.

In LoRA, we **freeze** W and add a trainable low-rank update:

$$W' = W + \Delta W$$

and LoRA defines:

$$\Delta W = BA$$

where

- $A \in \mathbb{R}^{r \times d_{\text{in}}}$
- $B \in \mathbb{R}^{d_{\text{out}} \times r}$

and r (rank) is a small number, e.g. 4, 8, 16.

So instead of learning a full $d_{\text{out}} \times d_{\text{in}}$ matrix,
we only learn two small matrices A and B .

Suppose we have:

- $d_{\text{in}} = 1024$
- $d_{\text{out}} = 1024$
- Full weight matrix W has $1024 \times 1024 = 1,048,576$ parameters.

Now LoRA uses $r = 8$.

Then:

- $A: 8 \times 1024 = 8,192$
- $B: 1024 \times 8 = 8,192$
- Total = $8,192 + 8,192 = 16,384$ parameters.

So instead of **1,048,576 parameters**,

LoRA trains **only 16,384 parameters**,

👉 a **64x reduction!**

During inference...

After fine-tuning, the model uses:

$$W_{\text{effective}} = W + \alpha \cdot (BA)$$

where α is a scaling factor (e.g. 1 or 0.1).

This can be **merged back** into W for efficiency,
so inference cost is **the same as the original model**.

LoRA – a PEFT

Approach	Parameters Updated	Example Params	Notes
Full Fine-Tune	All of W	1,048,576	High cost
LoRA	Only A & B	16,384	~64× fewer parameters

More conceptual understanding..

Step 1: Start with pretrained weight W

This is the big matrix already learned by the original model — it's **frozen** (not changed).

Step 2: Add a new learnable part ΔW

We introduce **two new small matrices** A and B , whose product gives ΔW .

These two matrices are **the only parts we train** during fine-tuning.

More conceptual understanding..

Step 3: Combine them during forward pass

During training, when the model does a forward pass:

$$h = (W + \Delta W)x = (W + BA)x$$

Here:

- W is frozen (pretrained)
- BA is the *low-rank update*
- Both are added together before multiplying by input x

So yes — the **low-rank matrix** BA is **learned separately** and then **added** to the frozen W during training and inference.

Let's say:

$$W : 1024 \times 1024$$

and LoRA rank $r = 8$

You create:

- $A : 8 \times 1024$
- $B : 1024 \times 8$

During training, you learn values for A and B .

The product BA gives a small "correction" matrix ΔW .

After training, you can even **merge** them back:

$$W_{\text{new}} = W + \alpha \cdot (BA)$$

Now you can discard A and B and just use W_{new} — no runtime overhead.

Step	Matrix	Status	Description
1	W	Frozen	Original pretrained weights
2	A, B	Trainable	Small new matrices (low-rank)
3	$\Delta W = BA$	Computed	Low-rank update added to W
4	$W' = W + \Delta W$	Effective weight	Used during training & inference

Ok,time to play with PEFT, LoRA... Coding time..

QLoRA: Quantization (Math Example)

Take a small weight vector:

$$W = [-1.30, 0.20, 0.75, 1.90, -0.60]$$

QLoRA commonly uses 4 bits so the integer range is $q \in [-8, 7]$ (symmetric). The per-tensor scale is

$$s = \frac{\max(|W|)}{2^{b-1} - 1} \quad \text{with } b = 4.$$

1. find $\max(|W|)$:

$$\max(|W|) = 1.90.$$

2. denominator for 4-bit: $2^{4-1} - 1 = 8 - 1 = 7$.

3. compute scale s :

$$s = \frac{1.90}{7}.$$

Do the division digit-by-digit:

$$1.90 \div 7 = 0.2714285714285714 \dots$$

So

$$s \approx 0.2714285714.$$

4. quantize each weight: $q = \text{round}(W/s)$.

Compute W/s for each element (I'll show the arithmetic and rounding):

- $W_1 = -1.30$: $-1.30 \div 0.2714285714 \approx -4.792$.
 $\Rightarrow q_1 = \text{round}(-4.792) = -5$.
- $W_2 = 0.20$: $0.20 \div 0.2714285714 \approx 0.73714$.
 $\Rightarrow q_2 = \text{round}(0.73714) = 1$.
- $W_3 = 0.75$: $0.75 \div 0.2714285714 \approx 2.7643$.
 $\Rightarrow q_3 = \text{round}(2.7643) = 3$.
- $W_4 = 1.90$: $1.90 \div 0.2714285714 = 7.0000$ (exact by construction).
 $\Rightarrow q_4 = 7$.
- $W_5 = -0.60$: $-0.60 \div 0.2714285714 \approx -2.2114$.
 $\Rightarrow q_5 = \text{round}(-2.2114) = -2$.

So the quantized integers are

$$q = [-5, 1, 3, 7, -2].$$

5. dequantize (reconstruct) with $\hat{W} = s \cdot q$:

Multiply s by each integer (digit-by-digit multiplications shown):

- $\hat{W}_1 = 0.2714285714 \times (-5) = -1.357142857$.
error = $\hat{W}_1 - W_1 = -1.357142857 - (-1.30) = -0.057142857$.
- $\hat{W}_2 = 0.2714285714 \times 1 = 0.2714285714$.
error = $0.2714285714 - 0.20 = 0.0714285714$.
- $\hat{W}_3 = 0.2714285714 \times 3 = 0.8142857143$.
error = $0.8142857143 - 0.75 = 0.0642857143$.
- $\hat{W}_4 = 0.2714285714 \times 7 = 1.900000000$ (exact).
error = 0.
- $\hat{W}_5 = 0.2714285714 \times (-2) = -0.5428571428$.
error = $-0.5428571428 - (-0.60) = 0.0571428572$.

W	W/s	q	$\hat{W} = sq$	error ($\hat{W} - W$)
-1.300000	-4.792000	-5	-1.357142857	-0.057142857
0.200000	0.737143	1	0.271428571	0.071428571
0.750000	2.764286	3	0.814285714	0.064285714
1.900000	7.000000	7	1.900000000	0.000000000
-0.600000	-2.211429	-2	-0.542857143	0.057142857

A neural network layer has the following (small) weight vector:

$$W = [-2.4, 0.8, 1.1, -0.5, 2.0].$$

You are asked to **quantize these weights to 4 bits** using **uniform symmetric quantization**.

Use the quantization formula:

$$s = \frac{\max(|W|)}{2^{b-1} - 1}, \quad b = 4$$

$$q = \text{round} \left(\frac{W}{s} \right)$$

$$\hat{W} = s \cdot q$$

Tasks

1. Compute the scale s .
2. Compute the quantized integers q .
3. Compute the reconstructed (dequantized) weights $\hat{W}$.
4. Compute the quantization error for each value:

$$\text{error} = \hat{W} - W.$$

$q = [-7, 2, 3, -1, 6]$

$\hat{W} = [-2.40, 0.6857, 1.0286, -0.3429, 2.0571].$

$W = [-2.4, 0.8, 1.1, -0.5, 2.0]$

$\text{error} = \hat{W} - W$

W	$\hat{W}$	error
-2.40	-2.40	0.0000
0.80	0.6857	-0.1143
1.10	1.0286	-0.0714
-0.50	-0.3429	0.1571
2.00	2.0571	0.0571

QLoRA: Quantization (Math Example)

- Repeat the previous problem in 8-bit and compare the error value.. What are your comments?
- Use MAE to compare two errors..

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n | \hat{W}_i - W_i |$$

Code: QLoRA

Prompt Tuning

A technique at the half way of prompt engineering and prompt tuning

It involves letting the model itself modify the embeddings that form the prompt, making it more efficient.

Prompt Tuning

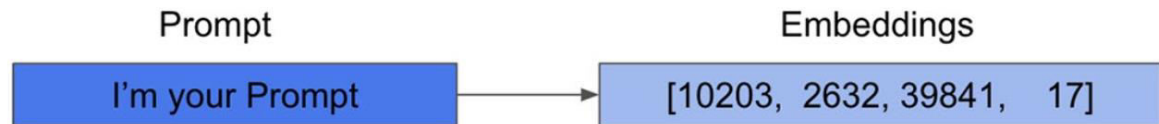


Figure 5-5a. From text to embeddings

Prompt Tuning

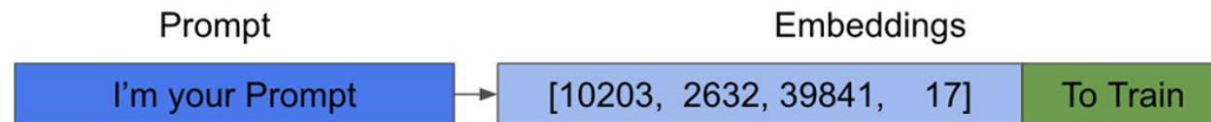
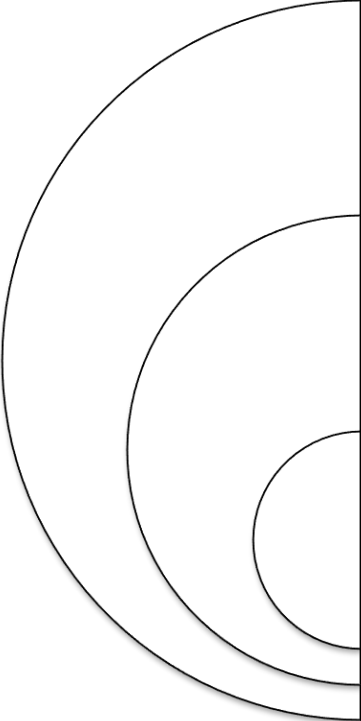


Figure 5-5b. Embeddings with additional space

Prompt Tuning



The model is creating a type of superprompt by enhancing a portion of the prompt with its acquired knowledge.

However, that particular section of the prompt cannot be translated into natural language.

It's as if the model mastered expressing in embeddings and generating highly effective prompts.

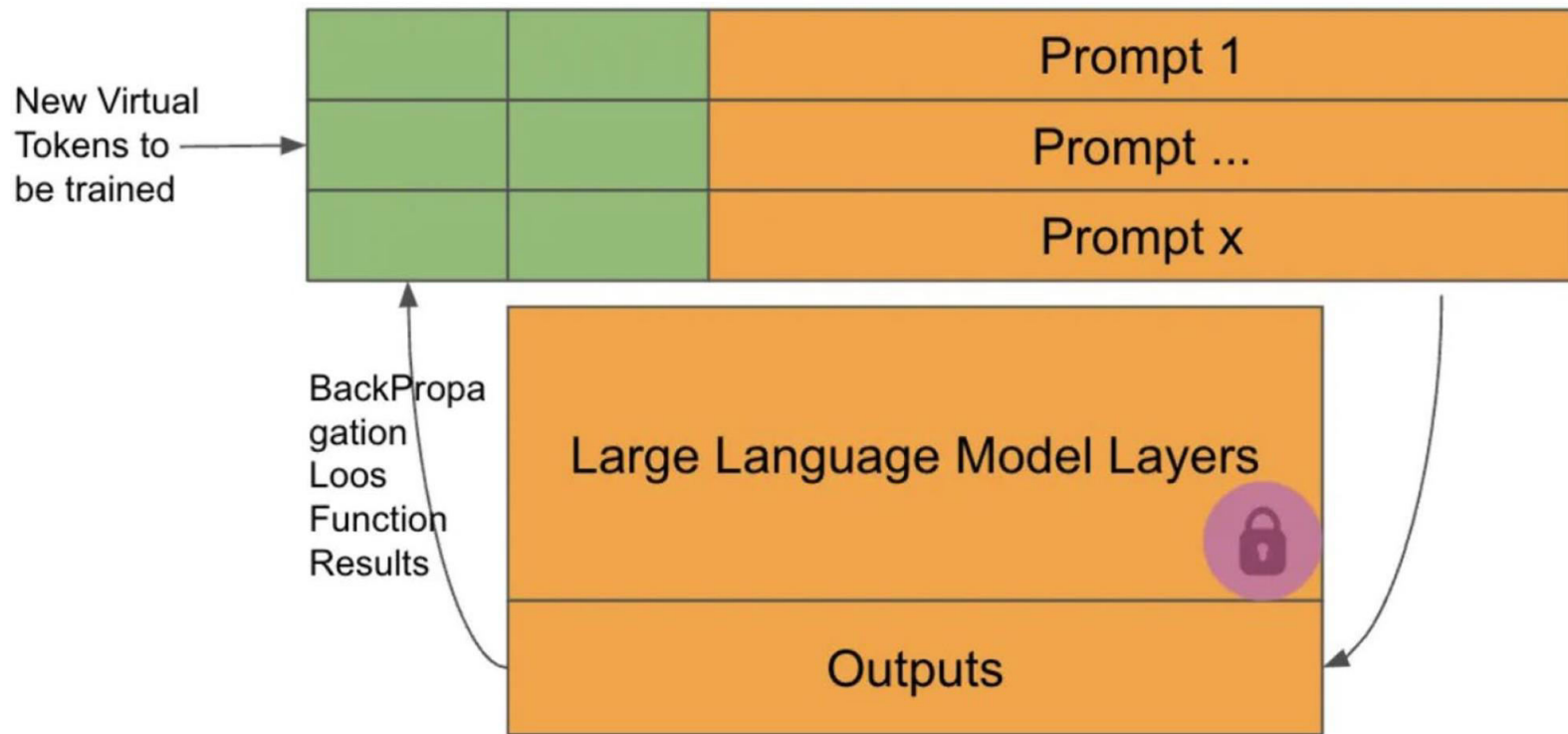


Figure 5-6. *Prompt-tuning overview*

Prompt Tuning

The most notable advantages of this form of fine-tuning are

- You train a ridiculous portion of the model's weights; in the notebook I use a Bloom model, and I am only training 0.0036% of the weights.
- You can have a single model in memory and several adapters (the trained layers that modify the prompt); the model's operation will be completely different depending on the adapter used.
- There is no catastrophic forgetting, or anything like it; the original model's weights are not touched.

Prompt Tuning

- Savings in inference cost. Forget about large prompts with multiple shot examples. The prompts generated in this way do not take up even a tenth of the space that a prompt created with prompt engineering does (each project will be different), but I have used 6 extra tokens per prompt to manage to adapt the model's behavior.

Prompt Tuning

However, it also has a drawback: It's not useful for giving new knowledge to the model. It's only useful for giving it a new personality or training it in a specific task.

